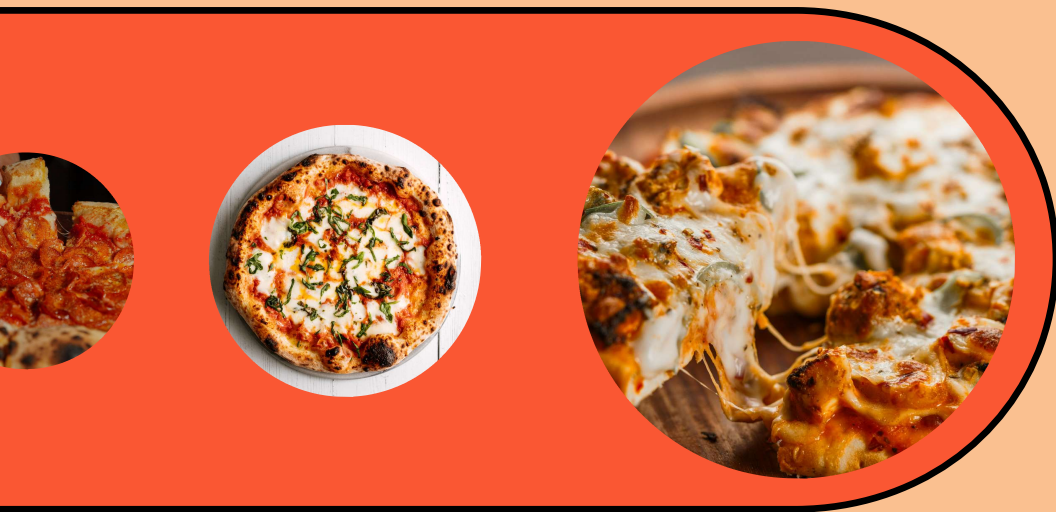


PIZZAHOME

EXPLORE PIZZAS





WELCOME OUR PIZZA

PizzaHome offers a variety of delicious pizzas in different sizes, categories, and flavors. Each pizza is available at a specific price and can be ordered in different quantities, making it easy to analyze customer preferences, popular pizza types, sales, and revenue.



OUR SERVICE



ORDER MANAGEMENT

PizzaHome provides efficient order management by maintaining details such as order date, time, pizza type, quantity, and order information. This helps organize orders and track daily sales easily.



SALES & ANALYTICS

PizzaHome provides sales analysis by calculating revenue, identifying popular pizzas and sizes, and analyzing category-wise and daily sales. These insights help the business make better data-driven decisions.

01.

PizzaHome is a pizza sales management and data analysis project developed using MySQL. It stores and manages information about orders, pizzas, pizza categories, prices, quantities, and order timings in a structured database.

02.

The project focuses on analyzing pizza sales data to generate useful business insights. It helps identify total orders, revenue, best-selling pizzas, popular sizes, category-wise sales, daily orders, and top pizzas based on revenue, supporting better data-driven business decisions.

OUR PROJECT



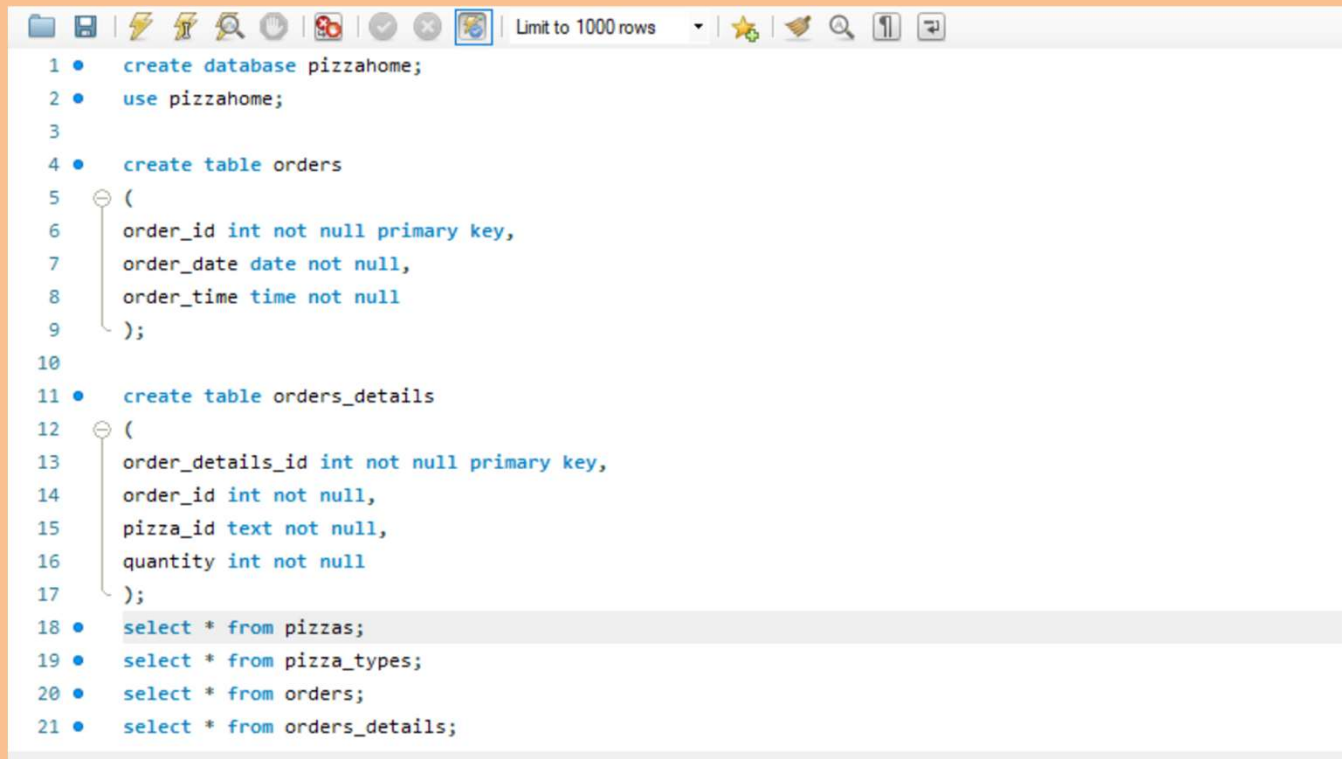


BEST PORTFOLIO

PizzaHome is a MySQL-based data analytics project that analyzes pizza orders and sales data to understand business performance and customer ordering patterns.

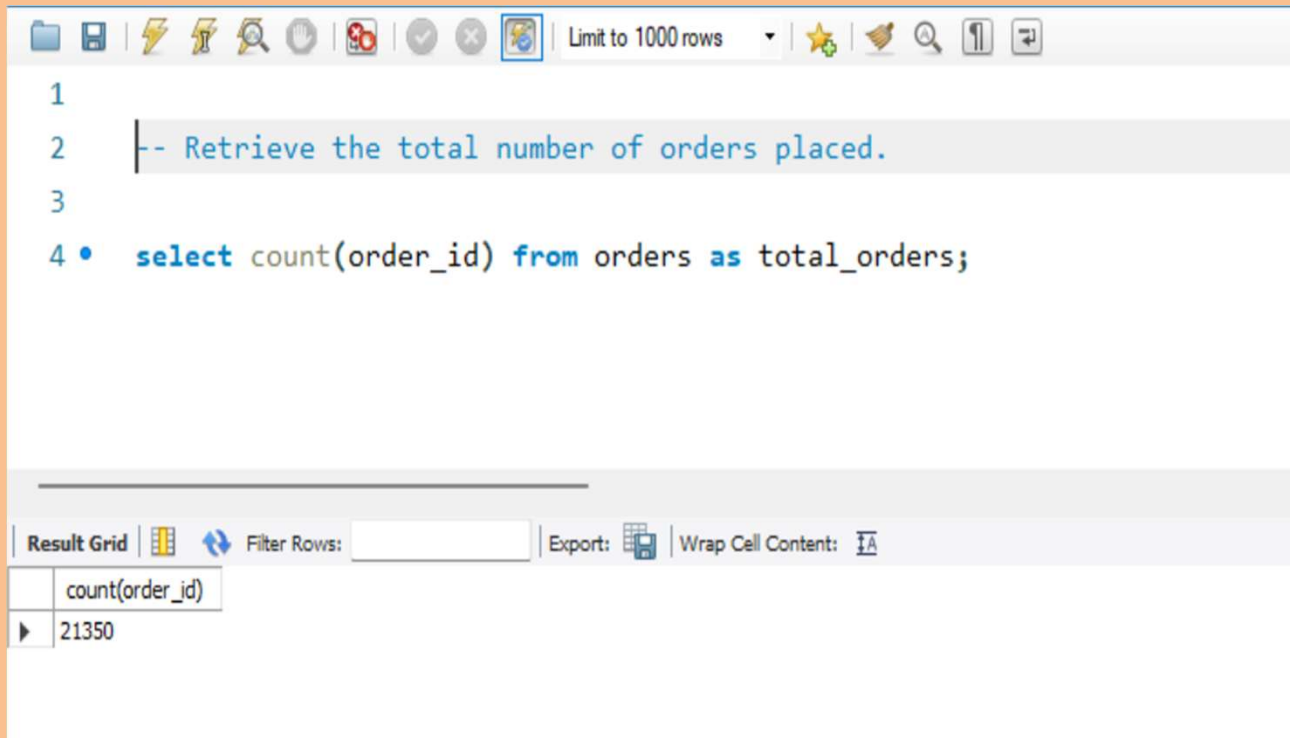
Key Analysis:

- Total number of orders and pizzas sold
- Total revenue generated
- Highest-priced pizza
- Most popular pizza size
- Top 5 best-selling pizza types
- Category-wise pizza sales
- Hourly and daily order trends
- Top pizzas based on revenue



The image shows a screenshot of a SQL IDE window. The window has a toolbar at the top with various icons for file operations, execution, and navigation. Below the toolbar, the SQL code is displayed in a monospaced font. The code consists of 21 lines, each preceded by a line number and a bullet point. The code creates a database named 'pizzahome', uses it, and then creates two tables: 'orders' and 'orders_details'. The 'orders' table has three columns: 'order_id' (int, not null, primary key), 'order_date' (date, not null), and 'order_time' (time, not null). The 'orders_details' table has four columns: 'order_details_id' (int, not null, primary key), 'order_id' (int, not null), 'pizza_id' (text, not null), and 'quantity' (int, not null). Finally, four SELECT statements are shown, each selecting all columns from a specific table: 'pizzas', 'pizza_types', 'orders', and 'orders_details'.

```
1 • create database pizzahome;
2 • use pizzahome;
3
4 • create table orders
5 • (
6 •     order_id int not null primary key,
7 •     order_date date not null,
8 •     order_time time not null
9 • );
10
11 • create table orders_details
12 • (
13 •     order_details_id int not null primary key,
14 •     order_id int not null,
15 •     pizza_id text not null,
16 •     quantity int not null
17 • );
18 • select * from pizzas;
19 • select * from pizza_types;
20 • select * from orders;
21 • select * from orders_details;
```



The screenshot shows a SQL IDE window with a toolbar at the top containing icons for file operations, execution, and navigation. A dropdown menu indicates 'Limit to 1000 rows'. The main text area contains a SQL query with line numbers 1 through 7. The query is a SELECT statement that calculates the total revenue by summing the product of quantity and price from the orders_details and pizzas tables, joined on pizza_id. The bottom section of the window displays the 'Result Grid' with a single column 'total_sales' and one row containing the value '817860.049999993'. The interface also includes options for 'Filter Rows', 'Export', and 'Wrap Cell Content'.

```
1
2  -- Calculate the total revenue generated from pizza sales.
3
4 • select
5  sum(orders_details.quantity * pizzas.price) as total_sales
6  from orders_details join pizzas
7  on pizzas.pizza_id = orders_details.pizza_id ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

total_sales
817860.049999993



Limit to 1000 rows



```
1
2  -- Identify the highest-priced pizza.
3
4 • select pizza_types.name, pizzas.price
5   from pizza_types join pizzas
6   on pizzas.pizza_type_id=pizza_types.pizza_type_id
7   order by pizzas.price desc limit 1;
```

Result Grid



Filter Rows:

Export: 

Wrap Cell Content: 

Fetch rows: 

	name	price
▶	The Greek Pizza	35.95

Limit to 1000 rows

```
1
2  -- Identify the most common pizza size ordered.
3
4 • select pizzas.size, count(orders_details.order_details_id) as o
5    from pizzas join orders_details
6    on pizzas.pizza_id = orders_details.pizza_id
7    group by pizzas.size order by o desc limit 1;
```

Result Grid   Filter Rows: Export:  Wrap Cell Content:  Fetch rows: 

	size	o
▶	L	18526

```
2 -- List the top 5 most ordered pizza types along with their quantities.
```

```
4 • select pizza_types.name, sum(orders_details.quantity) as quantity
5 from pizza_types join pizzas
6 on pizza_types.pizza_type_id = pizzas.pizza_type_id
7 join orders_details
8 on orders_details.pizza_id = pizzas.pizza_id
9 group by pizza_types.name order by quantity desc limit 5;
```

Result Grid		 Filter Rows:		Export:		Wrap Cell Content:		Fetch rows:	
	name	quantity							
▶	The Classic Deluxe Pizza	2453							
	The Barbecue Chicken Pizza	2432							
	The Hawaiian Pizza	2422							
	The Pepperoni Pizza	2418							
	The Thai Chicken Pizza	2371							

```
1  -- 6]Join the necessary tables to find the total quantity of each pizza category category.
2
3  •  select pizza_types.category,sum(orders_details.quantity) as total_quantity
4     from pizza_types join pizzas
5     on pizza_types.pizza_type_id = pizzas.pizza_type_id
6     join orders_details
7     on orders_details.pizza_id = pizzas.pizza_id
8     group by pizza_types.category order by total_quantity ;
9
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	category	total_quantity
►	Chicken	11050
	Veggie	11649
	Supreme	11987
	Classic	14888

The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains a query with a comment and a SELECT statement. Below the editor, the 'Result Grid' tab is active, displaying a table with two columns: 'hours' and 'total_order_count'. The table contains 8 rows of data. The status bar at the bottom indicates 'Result 1'.

```
1
2  -- Execute the selected portion of the script or everything, if there is no selection day.
3
4 • select hour(order_time) as hours, count(order_id) as total_order_count
5    from orders
6    group by hour(order_time);
7
8
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	hours	total_order_count
▶	11	1231
	12	2520
	13	2455
	14	1472
	15	1468
	16	1920
	17	2336
	18	2399

Result 1 x

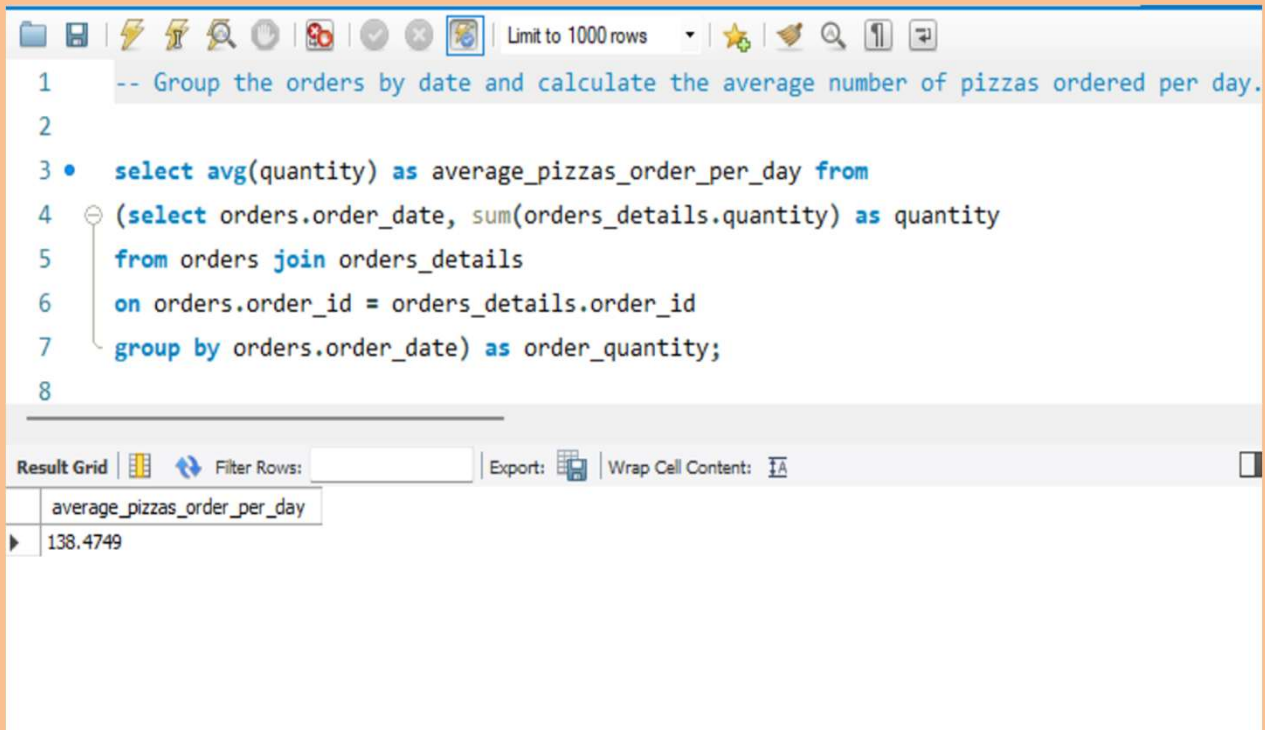
```
1
2  -- Join relevant tables to find the category-wise distribution of pizzas
3
4  • select category, count(name) from pizza_types
5     group by pizza_types.category;
6
7
```

Result Grid

	category	count(name)
▶	Chicken	6
	Classic	8
	Supreme	9
	Veggie	9

```
1
2  -- Group the orders by date and calculate the number of pizzas ordered per day.
3
4  •  select orders.order_date ,sum(orders_details.quantity)
5     from orders join orders_details
6     on orders.order_id = orders_details.order_id
7     group by order_date;
```

order_date	sum(orders_details.quantity)
2015-01-01	162
2015-01-02	165
2015-01-03	158
2015-01-04	106
2015-01-05	125
2015-01-06	147
2015-01-07	138
2015-01-08	173



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a dropdown menu set to "Limit to 1000 rows". The SQL editor contains the following code:

```
1  -- Group the orders by date and calculate the average number of pizzas ordered per day.
2
3  • select avg(quantity) as average_pizzas_order_per_day from
4    (select orders.order_date, sum(orders_details.quantity) as quantity
5     from orders join orders_details
6     on orders.order_id = orders_details.order_id
7     group by orders.order_date) as order_quantity;
8
```

Below the editor is the "Result Grid" section, which includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The results are displayed in a table:

average_pizzas_order_per_day
138.4749



THANK YOU